LAN Based Chat APP

A report submitted for the course named Project - I (CS3201)

Submitted By

SUYASH MISHRA
SEMESTER - V
220101077

Supervised By

DR. NAVANATH SAHARIA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY SENAPATI, MANIPUR

OCTOBER, 2024

Declaration

In this submission, I have expressed my idea in my own words, and I have adequately cited and referenced any ideas or words that were taken from another source. I also declare that I adhere to all principles of academic honesty and integrity and that I have not misrepresented or falsified any ideas, data, facts, or sources in this submission. If any violation of the above is made, I understand that the institute may take disciplinary action. Such a violation may also engender disciplinary action from the sources which were not properly cited or permission not taken when needed.

Suyash Mishra
220101077

DATE: __-Oct-2024

Department of Computer Science Engineering
Indian Institute of Information Technology Senapati, Manipur

Dr. Navanath Saharia
Assistant Professor, CSE

Email: nsaharia@iiitmanipur.ac.in
Contact No: +91 967-8073-826

To Whom It May Concern

This is to certify that the project/internsip report entitled **"LAN Based Chat APP",** submitted to the department of Computer Science and Engineering, Indian Institute of Information Technology Senapati, Manipur in partial fullfillment for the award of degree of Bachelor of Technology in Computer Science and Engineering is record bonafide work carried out by **Suyash Mishra** bearing roll number 220101077.

Signature of the Supervisor

Dr. Navanath Saharia

Signature of the Examiner 1 .....

Signature of the Examiner 2 .....

Signature of the Examiner 3 .....

Signature of the Examiner 4 .....

Abstract

The project involves the design and development of a LAN-based chat application that facilitates real-time communication within a local network using a hybrid approach with sockets and MQTT protocols. In today's connected world, regional communication within small networks plays a vital role in both personal and professional environments. While internet-based messaging platforms are widely available, a LAN-based chat application offers a critical solution in scenarios where internet access is restricted or unavailable. It ensures efficient communication within the local network, providing enhanced privacy by keeping all messages confined to internal systems, thereby minimizing the risks associated with external threats.

The project integrates sockets and MQTT for broadcasting messages and supporting group chats. This hybrid model enables flexibility, with direct messaging over sockets and multi-user communication via MQTT topics. Such an architecture ensures low-latency performance, making the application particularly useful in small-scale organizations, educational institutions, and closed groups where secure, internal communication is essential.

By eliminating the dependency on external networks, the chat application guarantees data security and gives users complete control over their communications. A user-friendly interface, combined with robust functionality, ensures reliability and ease of use. This project also serves as a hands-on exploration of networking concepts, protocol integration, and real-time communication, contributing valuable insights into the development of effective local communication systems.

Acknowledgement

I would like to express my deepest gratitude to Dr. Navanath Saharia, my project supervisor at Indian Institute of Information Technology Senapati, for his invaluable guidance, encouragement, and support throughout the development of this LAN-based chat application. His insights and expertise have been instrumental in shaping this project.

I am also immensely grateful to Dr. Nongmeikapam Kishorjit Singh, Head of the Department of Computer Science and Engineering, for providing a conducive environment and constant encouragement that made the completion of this project possible.

In addition, I extend my heartfelt thanks to my colleagues and the faculty members at IIIT Senapati, whose feedback and suggestions have helped improve various aspects of the project. Their input has been invaluable in refining the final product.

Lastly, I am deeply thankful to my family and friends for their continuous encouragement and support, which motivated me to complete this project successfully.

Suyash Mishra

Contents

List of Figures

Chapter 1

Introduction

This project focuses on developing a LAN-based chat application that allows users to communicate within a local network, offering an efficient, fast, and secure way to exchange messages. In today's interconnected world, effective communication is essential for both personal and professional interactions. While internet-based messaging platforms are widely used, LAN-based chat applications provide a critical alternative—especially in environments where internet access is restricted or unavailable. These applications enable real-time communication between users within the same network, ensuring data privacy and reducing the risk of external threats.

The primary objective of this project is to design and implement a group chat application that operates exclusively on a LAN, catering to small organizations, educational institutions, and closed communities, where internal communication plays an essential role. The application leverages socket programming for seamless peer-to-peer interactions, while MQTT enables multi-user messaging through shared topics, facilitating efficient group communication. This hybrid approach ensures low-latency messaging, robust performance, and flexibility for users.

By eliminating the need for internet connectivity, the application ensures that all communications remain private and under the control of the users within the local network. With an emphasis on user-friendly design, secure communication protocols, and reliable functionality, this project provides a practical solution for internal messaging. Additionally, the development process offers an opportunity to explore key concepts in networking, real-time communication, and security, making it a valuable contribution to the field of local communication systems.

1.1 Outline

[6],[7],[4]

1.1.1 Introduction

Outline

This project focuses on developing a LAN-based chat application for efficient, secure, and fast communication within a confined local network. LAN (Local Area Network) technology enables communication between devices without relying on the internet, offering a robust alternative for institutions or organizations with restricted or no internet access. By keeping all communication within the local network, the application ensures data privacy and mitigates the risks associated with external threats.

Problem Statement

The absence of efficient communication systems during internet outages creates challenges for institutions. In emergencies, institutions need an internal communication platform that does not rely on external networks. This project aims to address these issues by building a LAN-based chat application that ensures smooth communication within closed environments.

Motivation

In today's interconnected world, the need for effective internal communication is essential. LAN technology offers a reliable, fast, and private means of messaging. LAN chat applications play a vital role in offices, schools, and other organizations where privacy and network control are priorities, providing alternatives to internet-based communication tools that are prone to security risks.

Purpose

The primary goal of this project is to design and implement a chat application that operates within a LAN environment, ensuring data privacy, control, and real-time communication. It eliminates internet dependency, making it suitable for environments with limited or no internet access.

1.1.2 Literature Survey

Introduction

This section reviews the existing technologies, platforms, and research related to LAN-based chat applications. It also discusses the limitations of existing systems and the scope for improvement.

Existing Systems

This subsection describes current messaging systems, focusing on both internet-based and LAN-based tools. We will highlight popular chat tools such as Slack, Discord, and open-source LAN chat software.

Technology Review

A review of Java and Swing frameworks, emphasizing how they contribute to developing cross-platform, GUI-based applications, will be presented here.

Security Considerations

This section discusses the privacy challenges associated with internet-based messaging and how LAN-based chat applications can offer a safer alternative by operating within closed networks.

Gaps in Existing Systems

The shortcomings in current LAN chat systems are identified, such as poor GUI design, limited functionalities, or lack of support for multimedia messages, which this project aims to address.

1.1.3 Requirement Engineering

Purpose and Goals

Requirement engineering defines the scope and requirements for the application, aligning them with user expectations. The goal is to ensure that all essential functionalities are included in the final product.

Requirement Gathering Process

The requirements were gathered through interviews with students and staff, surveys on communication needs, and analysis of existing tools.

Functional Requirements

- User authentication
- Group chat support for collaborative discussions.
- File sharing functionality.
- Admin announcements to broadcast messages.

Non-Functional Requirements

- Performance: Minimal latency in message delivery.
- Security: End-to-end encryption for chat messages.
- Usability: Simple and intuitive GUI using Java Swing.
- Platform Independence: Java ensures cross-platform compatibility.

Constraints and Assumptions

The application must operate solely over LAN, without internet. It is assumed that users have access to LAN infrastructure, and administrators will manage the system.

1.1.4 System Design

Architecture Overview

The application follows a client-server architecture where the server handles message routing, and the clients communicate via the LAN.

Module Design

Server Modules

- **Messaging Module:** Manages the exchange of messages between clients, ensuring reliable and real-time communication over the LAN.
- **Admin Module:** Enables administrators to broadcast announcements to all users, facilitating the distribution of important messages.
- **File Transfer Module:** Oversees secure file transmission between users to ensure data integrity and proper delivery.

Client Modules

- **User Interface Module:** Built using Swing, this module handles user interaction, allowing users to send messages, view chats, manage files, and access settings.
- **Messaging Handler:** Connects with the server to send and receive messages seamlessly, ensuring smooth client-server communication.

Security Design

Encryption mechanisms ensure secure message delivery. User authentication is implemented to prevent unauthorized access.

1.1.5 Implementation

Development Environment

The application is developed using Java Development Kit (JDK) and the Swing framework for GUI. SQLite is integrated for data management.

Key Code Snippets

Relevant code snippets for the chat functionality, GUI rendering, and file transfer will be included here.

Testing Process

The application undergoes unit testing for individual modules and integration testing to ensure smooth communication between clients and the server.

1.1.6 Results

Functionality Testing

A summary of the features tested, including messaging, file transfer, and admin announcements.

Performance Evaluation

The application's performance is evaluated in terms of message delivery speed, resource usage, and scalability.

1.2 Problem Statement

In many small organizations, educational institutions, and localized environments, there is a need for reliable, secure, and fast communication without relying on internet-based services. While internet messaging platforms are widely available, they pose potential risks in terms of privacy, security, and reliance on external networks. Additionally, many environments may face limited or no internet connectivity, such as during government-imposed internet bans due to civil unrest, making external messaging platforms ineffective.

In such scenarios, the lack of a dedicated communication system for local network environments becomes critical, particularly when timely information dissemination is essential. For instance, educational institutions may need to make important announcements or updates when internet access is restricted. Existing LAN-based communication tools often lack modern features or fail to prioritize user-friendly design and security, which are vital for effective internal communication.

Thus, there is a pressing need for a LAN-based chat application that provides a private, secure, and efficient way for users to communicate within a closed network. This application should enable seamless real-time messaging, protect communication through encryption, and function without internet dependency, catering to the needs of organizations that prioritize internal data security and efficient local communication, especially during times of internet disruption.

1.3 Motivation

In today's fast-paced world, the need for effective and reliable communication has never been more crucial. As we rely on technology for daily interactions—whether in professional settings, educational institutions, or even casual conversations—issues surrounding connectivity, privacy, and security have emerged as significant concerns. Internet-based messaging platforms, while convenient and widely used, often come with limitations that can hinder effective communication in certain scenarios. This project aims to develop a LAN-based chat application that addresses these challenges by leveraging Local Area Network (LAN) technology.

1.3.1 The Significance of Local Area Network Technology

LAN technology connects devices within a confined geographic area, such as an office, school, or research facility. This intrinsic character-

istic of LAN allows for seamless communication without the need for an external network like the internet. By operating within a closed network, LAN systems enable users to share information quickly and securely, fostering an environment conducive to collaboration and productivity.

In environments such as offices, schools, military installations, and research facilities, where internal communication is paramount, LAN-based systems offer an essential solution. These systems ensure that sensitive data remains secure and contained, significantly reducing the risk of unauthorized access or data breaches. In addition to enhancing privacy, LAN communications can also provide faster and more efficient interactions due to reduced latencies associated with internet connections. As the number of devices connected to the network is limited, the potential for delays is minimized, resulting in a more responsive communication experience.

1.3.2 The Need for a Secure Communication Platform

The rise of traditional chat applications, which often rely on centralized servers or cloud-based systems, has inadvertently exposed users to a myriad of privacy risks and security vulnerabilities. Centralized systems can be susceptible to external attacks, data breaches, and other security threats, putting users' sensitive information at risk. For organizations where confidentiality is vital—such as educational institutions discussing exam results or companies sharing proprietary information—the potential ramifications of data breaches can be catastrophic.

In contrast, a LAN-based chat application operates solely within a closed network, providing a secure alternative to conventional messaging platforms. By ensuring that all communication takes place internally, organizations can safeguard their data against external threats while maintaining control over their communication channels. This localized approach not only enhances security but also empowers organizations to establish their communication protocols, reducing reliance on third-party services and the inherent risks that come with them.

1.3.3 Navigating Communication During Government-Ordered Internet Shutdowns

Another critical motivation for developing a LAN-based chat application is the challenge posed by government-imposed internet bans due to civil unrest or other emergencies. In various situations, such as during protests or political turmoil, governments may restrict or entirely

cut off internet access, leaving individuals and organizations without a means to communicate. This disruption can have severe consequences for timely information dissemination, especially in educational institutions where important announcements must be made.

In such instances, a LAN-based chat application can serve as a lifeline, enabling organizations to communicate effectively even when external connectivity is compromised. By creating a dedicated communication channel that functions independently of the internet, institutions can ensure that critical information reaches its intended audience in a timely manner, thereby enhancing overall operational efficiency.

1.3.4 Enhancing Internal Communication in Educational Institutions

Educational institutions, in particular, stand to benefit significantly from a LAN-based chat application. With a growing emphasis on collaborative learning and real-time information sharing, a secure and efficient communication platform is essential for fostering a conducive learning environment.

The ability to relay important announcements, facilitate discussions among students and faculty, and support group projects is vital for educational success. By providing a dedicated platform for internal communication, this project aims to enhance the educational experience, allowing for seamless interactions that support both teaching and learning objectives.

1.3.5 Exploring Networking Concepts and Security

Beyond addressing immediate communication needs, this project also presents an opportunity to delve into essential networking concepts, real-time communication technologies, and security practices. Understanding how to implement secure communication protocols and effectively manage network traffic are critical skills for aspiring IT professionals. This project not only serves as a practical solution for communication challenges but also as a valuable learning experience that equips developers with the knowledge and skills needed to navigate the complexities of modern communication systems.

1.3.6 Conclusion

In summary, the motivation behind developing a LAN-based chat application stems from the need for a secure, efficient, and reliable com-

munication platform that operates independently of internet connectivity. By leveraging LAN technology, this project seeks to address privacy concerns, mitigate security risks, and enhance internal communication for organizations, particularly in educational settings. As we continue to navigate an increasingly connected world, the ability to communicate effectively—regardless of external factors—will remain a fundamental necessity. This project not only aims to fulfill this need but also provides a practical foundation for understanding the vital role of technology in fostering collaboration and communication in local environments.

1.4 Purpose

The primary purpose of this project is to develop a LAN-based chat application that enables secure and efficient communication within localized environments. As organizations and educational institutions increasingly rely on technology for internal communication, the need for a reliable platform that operates independently of internet connectivity has become paramount. This application will leverage Local Area Network (LAN) technology to provide a dedicated messaging solution that addresses the challenges posed by internet dependence, particularly during situations where access is restricted or unavailable.

The chat application aims to ensure privacy and data security by operating within a closed network, reducing the risk of unauthorized access and data breaches that can occur with traditional internet-based messaging platforms. By prioritizing user-friendly design, the application will facilitate seamless real-time communication among users, allowing for the swift exchange of important information, announcements, and collaborative discussions.

Additionally, this project seeks to support institutions during emergencies, such as government-imposed internet bans or technical failures, by providing a reliable alternative for internal communication. By focusing on enhancing user experience and implementing robust security protocols, the application aims to foster an environment of effective communication and collaboration. Ultimately, this LAN-based chat application will serve as a practical tool for organizations and educational institutions, empowering them to maintain connectivity and ensure information flow, regardless of external factors that may disrupt internet access.

Chapter 2

Literature Survey

Given the gaps in current research and technology, this project aims to fill the need for a modern LAN-based chat application. It will not only provide essential messaging features but also focus on performance, usability, and security, making it a valuable tool for organizations, educational institutions, and other local setups. The insights from this literature survey will serve as a foundation for the design and implementation of the proposed system

2.1 Evolution of Chat Applications

The development of chat systems has evolved significantly over the past few decades, from basic text-based platforms to sophisticated multimedia messaging services. Early chat applications such as IRC (Internet Relay Chat) and AOL Instant Messenger paved the way for real-time communication. With the advent of internet-based messaging platforms, applications like WhatsApp, Slack, and Microsoft Teams became dominant tools for both personal and professional use.

However, the dependency on centralized servers or cloud-based systems brings challenges such as privacy risks, data breaches, and downtime issues. While internet-based messaging solutions are ideal for global communication, there are cases where local, closed-network communication is more beneficial, especially in controlled environments such as educational institutions and workplaces. This evolution highlights the need for specialized systems like LAN-based chat applications, which offer a private and secure alternative for communication within a local network.

2.2 LAN Technology and Its Role in Communication

A Local Area Network (LAN) connects devices within a limited geographical area, such as an office, school, or campus. LANs are essential for internal communication and data sharing, ensuring fast and reliable connections without the need for internet access. The key advantage of LAN technology is its ability to provide high-speed communication with minimal latency.

LAN-based chat applications exploit this infrastructure to facilitate real-time messaging while eliminating internet dependency. This feature is particularly important in institutions where data security is a priority or when internet access is restricted. Additionally, since LAN communication remains confined to the local network, it minimizes the risk of unauthorized external access, offering a private and controlled messaging environment.

2.3 Privacy and Security Issues in Internet-Based Communication

Privacy concerns are a growing issue with internet-based messaging platforms. Many popular messaging apps rely on cloud services or centralized servers to store user data, leaving it vulnerable to data breaches, unauthorized access, or cyberattacks. For example, high-profile incidents involving WhatsApp and Zoom have raised concerns about user privacy, prompting many institutions to seek safer communication alternatives.

In contrast, LAN-based chat systems store and manage data within the local network, providing a higher degree of control over sensitive information. These systems do not depend on external servers, making them less prone to outside attacks. Furthermore, implementing encryption protocols and user authentication mechanisms ensures that messages and files remain secure, even within a local setup. This approach makes LAN-based chat applications an ideal choice for sensitive environments such as military installations, hospitals, and educational institutions.

2.4 Use of LAN-Based Communication in Restricted Environments

LAN-based communication systems are particularly useful in environments where internet access is limited or unavailable. In government offices and defense sectors, LAN-based messaging is often used to maintain confidentiality and prevent unauthorized data leaks. Additionally, during periods of civil unrest or emergency situations, internet access may be restricted or suspended by government authorities to prevent the spread of misinformation.

In such scenarios, educational institutions and organizations can use LAN-based chat applications to continue essential communication and operations. For instance, during internet shutdowns, administrators can make important announcements, teachers can coordinate with students, and staff can communicate efficiently using the local network. This ensures that critical messages reach their recipients without relying on the internet, preventing disruptions in internal communication.

Chapter 3

Requirement Engineering

Requirement engineering ensures that the design and implementation of the LAN-based chat application align with user needs and system constraints. The system is developed using **Java** for backend logic and **Swing** for the graphical user interface. The primary focus is to build a secure, fast, and easy-to-use local messaging platform. This chapter outlines the purpose, requirements gathering, system needs, constraints, and the framework for change management.

3.1 Purpose and Goals of Requirement Engineering

The goal of requirement engineering is to gather, analyze, and validate the requirements for designing a chat application capable of seamless LAN-based communication. The specific objectives include:

- **Capture the Needs of Internal Users:** The application targets users such as students, faculty, or staff who require quick and reliable communication within institutions and organizations. It aims to address local messaging needs by facilitating easy, real-time interaction.
- **Leverage Java's Platform Independence:** Java ensures backend robustness by enabling the system to run consistently across various platforms (Windows, macOS, and Linux) without requiring changes to the core code.
- **Utilize Swing for Interactive GUI:** Swing provides a lightweight, desktop-friendly interface with essential interactive components, making it ideal for both laptops and desktops used in institutions.

- **Ensure Security and Privacy:** Since the application operates on a closed LAN, all communications are restricted to the local network, preventing unauthorized access and protecting sensitive information.
- **Provide Traceability from Requirements to Implementation:** A traceability mechanism ensures that all gathered requirements are implemented and linked to specific project deliverables, maintaining quality assurance throughout the development.

3.2 Requirement Gathering Process

The requirements for the chat application were collected through various methods:

- **Interviews:** Conversations with students, staff, and administrators were conducted to understand their messaging needs, including preferences for features like announcements and file sharing.
- **Surveys:** Questionnaires were distributed to capture feature priorities, such as group chat, file sharing, and status indicators, allowing users to express what they consider essential.
- **Observation:** Current tools in use (e.g., email and LAN messaging) were analyzed to identify usability issues, performance gaps, and unmet needs that this chat application could address.
- **Review:** Existing chat applications, both open-source and proprietary, were studied to understand strengths, limitations, and design practices that could influence this project.

3.3 Functional Requirements

The following functional requirements define the core operations of the chat application:

1. **User Authentication:** Users must log in with valid credentials to ensure only authorized users access the system.
2. **Group Chat Support:** Users should have the ability to create and participate in group discussions, enabling collaboration and multi-party communication.

3. **File Sharing Capability:** Users must be able to share files (e.g., PDFs, images) through the chat interface, ensuring smooth exchange of resources.

4. **Admin Announcements:** Admins should have the capability to broadcast important messages to all users, ensuring timely dissemination of announcements.

3.4 Non-Functional Requirements

The non-functional requirements ensure the chat application performs efficiently and securely:

- **Performance:** The system must minimize latency so that messages are delivered without noticeable delays, even with multiple users active.
- **Security:** Data encryption should be used to protect message content during transmission, and authentication measures should prevent unauthorized access.
- **Usability:** The GUI should be intuitive, using Swing components to offer an easy and smooth user experience, even for non-technical users.
- **Scalability:** The application should handle multiple users seamlessly, ensuring performance does not degrade with an increasing number of participants.
- **Platform Independence:** Since the application is built in Java, it must run smoothly across various operating systems without requiring code changes.

3.5 System Requirements

Hardware Requirements:

- A server machine is required to host the backend services that manage user data, messaging, and other functionalities.
- Client devices (desktops or laptops) should have Java Runtime Environment (JRE) installed to run the application.
- A router or switch is needed to set up the LAN infrastructure for network communication.

Software Requirements:

- **Java Development Kit (JDK) 17+** for writing and compiling the backend logic and GUI.
- **Swing** library to create the graphical user interface for interaction.

3.6 Constraints and Assumptions

Constraints:

- The application must function exclusively on the LAN without internet access, ensuring local communication only.
- Development resources and budget are limited, restricting extensive feature expansion.

Assumptions:

- Users will have access to the LAN network at all times when using the application.
- Administrators will be responsible for managing user access, group creation, and announcements.

3.7 Use Case Analysis

Group Chat:

- **Actors:** Multiple users, including regular users and admins.
- **Description:** An admin creates a group, and users join to exchange messages. Group members can share files and participate in collaborative discussions within the LAN environment.

3.8 Requirement Validation and Verification

Validation:

- Regular feedback sessions with project supervisors ensure the application aligns with intended requirements.

- Approval from **Dr. Navanath Saharia** and **Dr. Nongmeikapam Kishorjit Singh** ensures the project meets academic and technical expectations.

Verification:

- A traceability matrix is maintained to map each requirement to its corresponding implementation for tracking progress.
- User acceptance testing (UAT) ensures that the final product meets end-user needs and performs as expected.

3.9 Change Management Process

- **Change Requests:** Users or stakeholders can submit requests for new features or modifications.
- **Impact Analysis:** Changes are analyzed to assess their impact on the project's scope, cost, and schedule.
- **Approval Process:** Supervisors must approve all changes before they are implemented to maintain project alignment.
- **Documentation:** All changes are documented, and the requirements document is updated accordingly to ensure traceability.

3.10 Conclusion

Requirement engineering for the LAN-based chat application ensures the system meets user needs with a focus on security, privacy, and performance. Java and Swing provide a cross-platform solution with a responsive, intuitive interface. Proper validation and change management frameworks guarantee the project stays aligned with stakeholder expectations throughout development, resulting in a reliable and efficient communication platform.

Chapter 4

System Design

The system design of the LAN-based chat application is centered around a client-server architecture to facilitate real-time communication within a local network. The server acts as a central hub, managing connections and message exchanges between multiple clients. Each client connects to the server over a TCP socket, ensuring reliable data transmission. To support simultaneous communication, the system leverages multi-threading, enabling the server to handle multiple clients concurrently without blocking. This modular design ensures scalability and fault tolerance, as the server remains operational even if one or more clients disconnect unexpectedly.

The architecture focuses on simplicity, ensuring that messages are routed efficiently between users with minimal latency. Additionally, a graphical user interface (GUI) on the client side enhances usability, making it easier for users to exchange messages intuitively. This section explores the core modules—server, client, and network communication—and provides insights into the technologies and protocols employed to build the system.

4.1 Server Module: Handles Client Connections and Message Routing

As a rule, the activity of any server is to offer a unified assistance. Be that as it may, there are a wide range of methods for offering types of assistance, and various approaches to structure the interchanges. Visit is generally portrayed as an

Chat Server Interaction

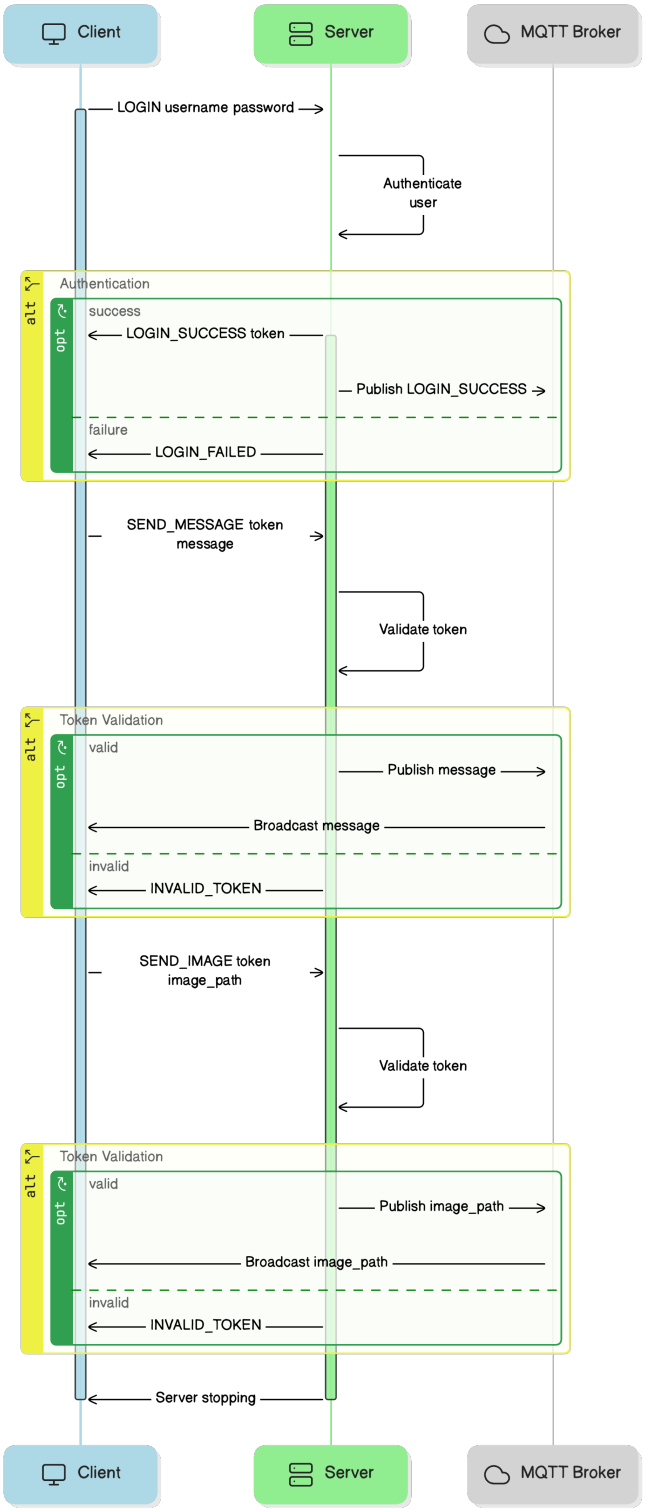

Figure 4.1: Chat Server Interaction Diagram

association, sending and getting content for the term of the meeting.[1] This module utilizes socket programming to establish reliable connections with multiple clients, employing the TCP protocol for stable, loss-free message delivery.

Additionally, the server integrates the MQTT protocol to enable group messaging through shared topics, allowing seamless communication among multiple users and efficient message broadcasting. This combination ensures low-latency performance and supports both direct peer-to-peer exchanges and multi-user interactions within the local network.

4.1.1 Socket Initialization and MQTT-Enabled Connection Handling

The server initiates a TCP socket on a specified IP address and port, listening for incoming connection requests from clients. Once a client connects, the server establishes a dedicated connection by accepting the client's request. The use of multi-threading ensures that each client connection is handled independently, preventing one client's message from blocking others. For every connected client, a new thread is spawned to manage its communication, allowing simultaneous interaction with multiple clients without performance bottlenecks.

In addition to handling individual client connections, the server integrates the MQTT protocol to support group messaging through shared topics, enabling efficient broadcasting and multi-user interaction. This hybrid approach ensures smooth communication across both direct peer-to-peer connections (via sockets) and group chats (via MQTT).

In case of unexpected disconnections or errors, the server module implements error-handling mechanisms to close inactive sockets, free resources, and disconnect from MQTT topics where needed. This ensures that the system remains stable, responsive, and ready to accept new clients or manage new topics without interruption.

4.1.2 Login Functionality Using JWT Token

The login functionality of the LAN-based chat application employs JSON Web Tokens (JWT) to ensure secure authentication and authorization of users. When a user attempts to log in, they provide their credentials (username and password) through the client module's user interface. Upon submission, the client sends these credentials to the server over a secure connection.

The server verifies the provided credentials against the stored user data. If the credentials are valid, the server generates a JWT that includes essential user information, such as the user ID and roles, and assigns an expiration time to the token. This token is then sent back to the client, where it is securely stored, typically in memory or local storage.

4.1.3 Maintaining Active Client Connections

The server keeps track of connected clients using a data structure, such as a dictionary or a list, where each client is associated with its socket and address. This client registry allows the server to broadcast messages efficiently by iterating over active connections. Additionally, client-specific information, such as usernames can be managed to enhance the user experience.To ensure that the server remains responsive during peak usage. This prevents the system from being overwhelmed by idle connections, optimizing performance.

4.1.4 Message Routing and Broadcasting

One of the core responsibilities of the server module is message routing. When a client sends a message, the server receives it, determines the intended recipient(s), and forwards the message accordingly. For group chats or public chat rooms, the server broadcasts the message to all connected clients. This involves sending the same message packet to every active client socket, ensuring that the conversation is shared across participants.

In scenarios where the chat application supports private messaging, the server uses targeted routing to deliver messages to specific clients only. The message is transmitted directly to the recipient's socket, ensuring confidentiality.

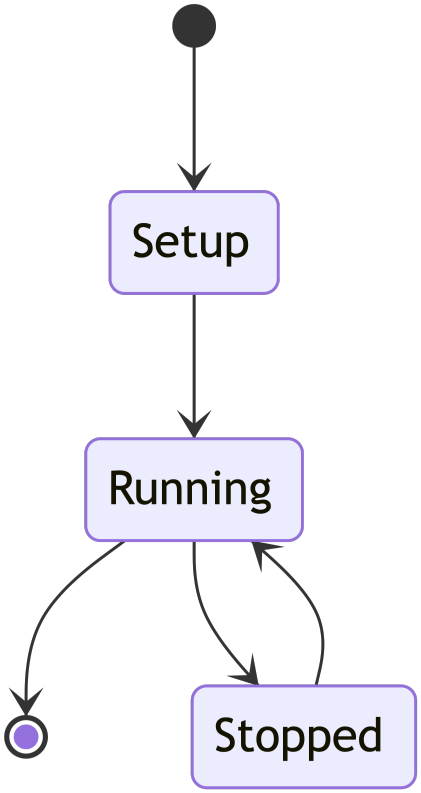

Figure 4.2: Server State Diagram

4.1.5 Concurrency Management and Synchronization

Managing multiple client connections requires effective concurrency handling. The server uses thread synchronization techniques to prevent race conditions, especially when multiple threads attempt to access shared resources, such as the client list or message logs. Mutexes or locks are used to ensure that operations, such as adding or removing clients, are executed safely without conflicts.

4.1.6 MQTT Integration in the Server Module for Group Communication

MQTT is the preferred loT publish-subscribe lightweight messaging protocol. If you develop IoT, web applications, mobile apps, or a combination of these solutions, you must learn how MQTT and

its lightweight messaging system work. Of course, it is extremely important to take security into account when working with this protocol.[3]

Learning MQTT is challenging because it includes too many abstract concepts that require real-life examples to be easy to understand.

The MQTT protocol plays a crucial role in the server module by enabling efficient group communication and message broadcasting. MQTT is a lightweight, publish-subscribe messaging protocol designed for real-time data exchange, making it ideal for scenarios where multiple users need to interact simultaneously. In this chat application, the server acts as both a publisher and broker, managing topics to which clients can subscribe.

This allows users to send messages to shared topics, facilitating group chats without the need for direct peer-to-peer connections between all participants. MQTT's low bandwidth consumption and asynchronous communication model ensure that messages are delivered promptly, even under constrained network conditions. Additionally, the protocol's quality of service (QoS) levels ensure reliable message delivery, providing flexibility to balance performance and reliability. By integrating MQTT, the server module supports scalable, low-latency communication and seamless interaction among multiple clients, enhancing the overall efficiency of the LAN-based chat system.

4.2 Client Module: Sends/Receives Messages and Displays the Chat Interface

The client module forms the user-facing component of the LAN-based chat application, responsible for establishing communication with the server, sending and receiving messages, and providing an intuitive graphical user interface (GUI) for seamless interaction. This module connects to the server over a TCP socket for peer-to-peer communication and utilizes MQTT to participate in group chats via shared topics. The client module enables users to engage in real-time chat sessions with other participants on the network, supporting both direct messaging and group communication.

With a focus on usability, responsiveness, and reliability, the client module ensures a smooth messaging experience while also man-

aging network events, such as disconnections or reconnections. Through MQTT, the client can subscribe to and publish messages in relevant group topics, allowing for asynchronous interaction without blocking direct messages. This hybrid approach ensures that users experience both low-latency communication and scalable group interaction, enhancing the overall performance of the chat application.

4.2.1 Login Functionality Using JWT Token

Subsequent requests made by the client to the server include the JWT in the Authorization header, allowing the server to authenticate the user without needing to revalidate the credentials each time. The server decodes the JWT, checks its validity and expiration, and grants access to the requested resources based on the user's permissions.[2]

Using JWT for login functionality provides several advantages, including stateless authentication, scalability, and enhanced security. By reducing server-side session management, the application can efficiently handle multiple users while maintaining a robust security framework. This design ensures that users can securely log in and access the chat features while keeping sensitive data protected.

4.2.2 User-Selected Protocol Preference During Login

The LAN-based chat application offers users the flexibility to select their preferred communication protocol during the login process. Upon launching the application, users are presented with options to choose between TCP sockets for direct messaging or MQTT for group communication. This choice allows users to tailor their chat experience based on their specific needs and the nature of their interactions.

For instance, users who primarily engage in one-on-one conversations may prefer the reliability of TCP sockets, while those who participate in group discussions can opt for the efficiency of MQTT. Once the user selects their preferred protocol and logs in with their credentials, the application establishes the necessary connections accordingly, ensuring a seamless and personalized messaging experience. This design enhances user satisfaction by

accommodating different communication styles and preferences within the same application framework.

4.2.3 Socket Initialization and Server Communication

When the client application starts, it initializes a TCP socket and attempts to connect to the server using the server's IP address and port. If the connection is successful, the client becomes part

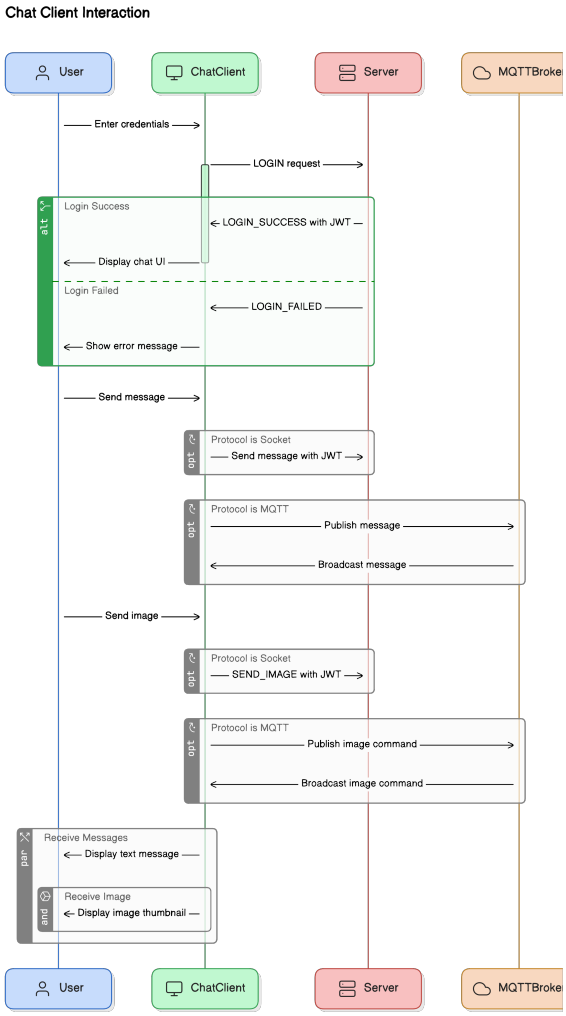

Figure 4.3: Chat Client Interaction Diagram

of the chat network. A handshake process with the server ensures that the client is registered, and its details (such as username or session ID) are logged by the server to facilitate further communication.

In addition to the socket connection, the client establishes MQTT subscriptions to relevant topics for group chat functionality, enabling it to participate in shared conversations. The client continuously listens for messages from both the server (via socket) and group topics (via MQTT) using dedicated threads, ensuring that neither communication stream blocks the other. This threaded design allows messages to be displayed in real-time without freezing the GUI, even when multiple messages arrive in quick succession. This hybrid communication model ensures a smooth, responsive messaging experience for users, supporting both individual and group interactions seamlessly.

4.2.4 Sending Messages

When a user types a message and presses the send button, the client module encapsulates the message along with metadata (such as sender name and timestamp) into a packet. This packet is then transmitted to the server through the established TCP socket. The server routes the message to other clients, ensuring that it reaches its intended recipient(s).

The sending mechanism incorporates input validation to ensure that empty or invalid messages are not transmitted. The client can also publish it to the relevant MQTT topic, allowing all users subscribed to that topic to receive the message simultaneously. Additionally, some chat interfaces allow for multi-line messages or formatting options, requiring the client module to handle such complexities gracefully before transmission. This includes properly formatting the message content and ensuring that any special characters or formatting tags are correctly encoded. By addressing these aspects, the client module enhances the overall user experience, ensuring that messages are sent accurately and effectively.

4.2.5 Receiving and Displaying Messages

Once the server broadcasts or sends a message, the client's listener thread receives the data packet and updates the chat interface. The client module ensures that received messages are rendered with proper formatting, including sender names, timestamps, and other UI elements like avatars (if applicable).

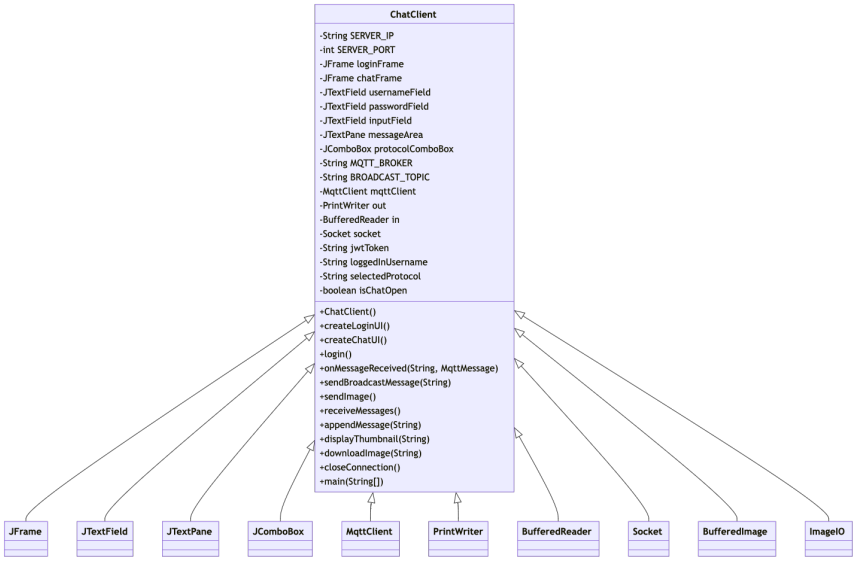

Figure 4.4: ChatClient Class Diagram

Messages from different users are typically displayed in separate styles (e.g., different colors or alignments) to distinguish between participants in the chat.

If the application supports private or group messaging, the client module identifies the message type and displays it accordingly. Group messages appear in a shared chat window, while private messages might trigger notifications or open a separate conversation window.

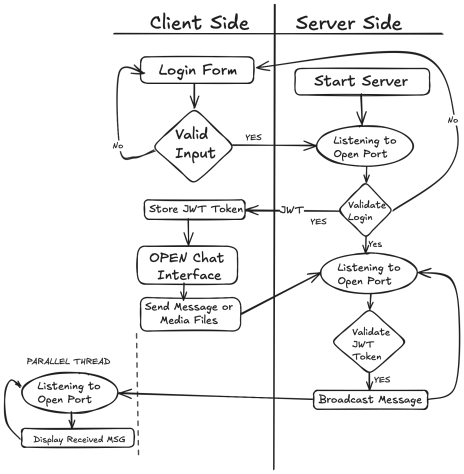

Figure 4.5: Swim Lane Diagram explaining Client and Server Modules

Chapter 5

Implementation

The client module of the LAN-based chat application was developed using Java Swing to create an intuitive and responsive graphical user interface (GUI). Java Swing provides a robust framework for building cross-platform desktop applications, making it an ideal choice for crafting the client-side interface. This section describes how key components, including socket communication, MQTT integration, GUI elements, and multi-threading, were implemented to ensure smooth message exchange and a seamless user experience.

5.1 Socket Initialization and Connection to Server

To enable communication with the server, the client establishes a TCP socket using Java's Socket class. The client connects to the server by specifying the server's IP address and port. Once connected, the socket is used to send and receive messages in real time.

5.2 MQTT Connection Establishment and Group Communication

The client module establishes an MQTT connection to facilitate group communication within the LAN-based chat application. Upon startup, the client connects to the MQTT broker, using the server's IP address and designated MQTT port. This connection allows the client to subscribe to relevant topics for group chats, enabling it to receive messages broadcasted by other users in real-time.[5]

When a user sends a message intended for a group, the client publishes it to the appropriate MQTT topic, ensuring that all subscribed clients receive the message simultaneously. This integration of MQTT enhances the chat experience by providing a lightweight, efficient, and scalable solution for multi-user interactions, allowing users to engage in dynamic group discussions without overloading the system or experiencing delays. By utilizing the publish-subscribe model, the client effectively manages message dissemination, keeping communication seamless and responsive.

5.3 Multi-Threading for Message Reception

The client application uses a separate thread to listen for incoming messages from the server continuously. This ensures that the main GUI thread remains responsive and does not freeze when receiving multiple messages.

The startListening method creates a new thread that constantly reads incoming messages from the server and updates the chat interface in real-time.

5.4 GUI Implementation using Java Swing

The graphical user interface (GUI) for the LAN-based chat application was implemented using Java Swing, a lightweight framework for building platform-independent desktop applications. The goal was to create an intuitive and user-friendly interface that allows seamless communication within a local network. Swing components such as JFrame, JTextArea, JTextField, JButton, and JScrollPane were utilized to design the core elements of the chat window, including a chat display area, message input field, and a send button.

The layout was managed using BorderLayout to position components effectively, with the chat area placed at the center and the input field and send button arranged at the bottom for easy access. Key event listeners were added to ensure smooth interaction; clicking the send button or pressing the Enter key triggers message submission.

To make the chat area more readable, the JTextArea was set to be non-editable and wrapped to fit long messages, while a JScroll-

Pane ensures the chat display remains accessible even for extended conversations. The GUI elements were initialized on the Event Dispatch Thread (EDT) using SwingUtilities.invokeLater() to ensure thread safety, following best practices for Java Swing development.

This GUI forms the foundation for the chat client. Once the user enters a message, it gets appended to the chat display, simulating a conversation. In addition, Socket programming will be integrated to facilitate real-time communication by sending messages between connected clients and servers over the LAN. The design focuses on simplicity, responsiveness, and ease of use to provide an efficient communication tool in scenarios where internet-based chat applications are unavailable or restricted.

5.5 Handling Disconnections and Errors

To ensure a reliable experience, the client module gracefully handles disconnections. If the server disconnects or the network connection fails, the client displays an error message and automatically attempts to reconnect, ensuring minimal disruption to the user's chat experience.

Implemented using Java Swing, the client module provides a smooth and interactive user interface for real-time chat within a LAN. It utilizes TCP sockets for reliable communication and integrates MQTT for efficient group messaging, allowing users to participate in both direct and group chats seamlessly. Multithreading is employed to handle incoming messages without blocking the GUI, ensuring a responsive experience even during periods of high activity. With components like JTextArea, JTextField, and JButton, the interface remains intuitive and user-friendly. Comprehensive error handling guarantees that the client remains stable during disconnections, making it a robust solution for peer-to-peer communication and group interactions alike.

Chapter 6

Results

The testing phase of the LAN-based chat application demonstrated the successful implementation of core functionalities, stable communication, and a user-friendly interface. Below are key results observed during testing:

6.1 Connection and Communication

The client module successfully established connections to the server using TCP sockets and the MQTT protocol, ensuring seamless communication across various scenarios. Multiple clients could connect concurrently without any conflicts or delays. Each connection was logged accurately on the server side, including the relevant IP addresses and session information. Once connected, clients could send messages to the server, which routed them either as group messages or private messages, depending on the context.

In group chats, the server utilized the MQTT protocol to efficiently broadcast each message to all connected clients, ensuring that participants received them in real time. For private messaging, the server correctly routed messages to the intended recipient without any leaks to other clients, leveraging TCP for reliable direct connections. The dependable TCP connection ensured that no messages were lost or corrupted during transmission. Even with multiple clients sending messages simultaneously, the server handled the load efficiently. This validated the system's capacity to maintain smooth communication across various messaging scenarios, including one-to-one and one-to-many interactions.

6.2 Real-Time Performance

The chat application demonstrated real-time message delivery, with negligible latency observed during testing. As soon as a client sent a message, it appeared on the chat interface of all connected users without delay. This was made possible through multi-threading, where a dedicated thread on each client handled incoming messages. The separation of tasks ensured that the user interface (UI) remained responsive even when processing a high volume of messages.

The smooth performance was consistent throughout different scenarios, including rapid exchanges in group chats. The use of a local network contributed to minimal lag, and the system maintained its efficiency with multiple active participants. During stress testing, the interface handled continuous messaging from several users without freezing or becoming unresponsive. This confirmed the system's capability to support concurrent usage, ensuring that real-time communication needs were met even under high activity, making it ideal for LAN-based environments.

6.3 Error Handling and Disconnection Management

The application included robust error handling and disconnection management mechanisms to ensure reliable operation. When a client disconnected—either intentionally or due to network issues—the server promptly detected the disconnection and released the associated socket resources. This prevented memory leaks or lingering inactive connections. If a client reconnected, the system resumed smoothly, allowing users to continue conversations without needing to restart the application.

Additionally, input validation was implemented to prevent invalid data, such as empty or excessively long messages, from being transmitted. This enhanced usability by reducing unintended errors during communication. The client module provided appropriate feedback in case of connectivity failures, such as displaying error messages when the server was unreachable. Automatic reconnection attempts were also tested, and the system proved capable of reconnecting seamlessly after temporary network disruptions, ensuring minimal downtime. These error management

features made the application reliable for real-time messaging in dynamic LAN environments.

6.4 User Interface and Usability

The Java Swing interface provided a simple and functional user experience, focusing on clarity and ease of use. The chat window, implemented using JTextArea and wrapped in a JScrollPane, ensured smooth scrolling through long message histories. Incoming messages were displayed with sender names and timestamps, making it easy for users to follow the conversation. Sent messages were differentiated using formatting techniques, such as text alignment or color changes.

The input field and send button allowed users to compose and transmit messages efficiently. The interface responded quickly to user actions, with the input field clearing immediately after each message was sent. Users found the GUI intuitive and minimalistic, contributing to easy navigation even during long chat sessions. The layout remained consistent across different window sizes, ensuring a pleasant user experience. These features made the chat application accessible and user-friendly, even for individuals with minimal technical knowledge.

Chapter 7

Conclusion

The development of a LAN-based chat application was a valuable exercise in creating a real-time communication tool using Java and Java Swing. This project not only demonstrated the core concepts of client-server architecture but also integrated network programming, multi-threading, and error handling mechanisms. This section summarizes key insights from the project, highlights the challenges that were overcome, and discusses potential improvements and future directions. Below are four major takeaways from the project:

7.1 Achieving Reliable Communication through Client-Server Architecture

The project successfully implemented reliable client-server communication using TCP sockets. The modular design facilitated smooth interaction between the server and multiple clients, with effective message routing managed by the server. Both group communication and private messaging functionalities were seamlessly achieved, validating the robustness of the underlying communication protocol.

The use of TCP sockets ensured that all messages were reliably delivered without loss or corruption, making it suitable for real-time scenarios. By employing synchronous message passing, the server could either broadcast messages to all clients or send them directly to specific users as needed. The project emphasized the importance of concurrency management on the server side to handle simultaneous client connections effectively. Overall, the

architecture laid a solid foundation for the development of more complex messaging systems, such as multi-room or role-based chat applications.

7.2 Overcoming Real-Time Performance Challenges with Multi-Threading

Achieving real-time performance required careful design to ensure that the system remained responsive under varying workloads. One of the key technical challenges was preventing the user interface from freezing while processing incoming messages, which was addressed by employing multi-threading on both the client and server sides.

In the client module, separate threads were utilized to listen for incoming messages, ensuring that the main GUI thread remained responsive at all times. On the server side, multi-threading enabled the simultaneous handling of multiple client connections without blocking the processing of other requests. During performance testing, the application maintained smooth communication even with multiple users sending messages continuously, confirming the effectiveness of the thread management strategy. This experience underscored the importance of balancing concurrency while ensuring that shared resources, such as network sockets, were properly synchronized to avoid conflicts or data inconsistencies.

7.3 Robust Error Handling and Reliable Disconnection Management

Error handling was another critical area of focus in the project. Given the nature of network-based applications, unexpected disconnections and network failures were inevitable. The project implemented strategies to detect and manage such scenarios gracefully. On detecting a disconnection, the server released the associated resources, preventing potential memory leaks or dangling connections. The client module also provided reconnection capabilities, allowing users to resume communication without restarting the application.

Input validation played a significant role in enhancing usability by preventing invalid data transmissions, such as empty or excessively long messages. In addition, the system provided feedback to users through error messages, making it easier to understand and resolve issues. These measures ensured that the application could handle real-world usage scenarios, where network instability or abrupt user exits might occur. The focus on robust error handling made the system more reliable and resilient, even under challenging conditions.

7.4 User Experience and Scope for Future Enhancements

The Java Swing-based GUI provided a functional, user-friendly interface for communication. While the project successfully delivered the core messaging features, several areas for improvement were identified during the testing phase. One potential enhancement is to implement a more modern UI framework, such as JavaFX, to offer a richer visual experience. Adding message formatting options like bold, italics, and emojis could further improve the usability and engagement of the chat system.

Future development could also explore features such as file sharing, chat room creation, or encryption for secure messaging. The current system works effectively within a LAN environment, but extending it for internet-based communication would require modifications to support firewalls, NAT traversal, and security protocols. Moreover, the architecture allows for the integration of additional communication protocols in the future, enhancing flexibility and functionality. Additionally, offline messaging capabilities could enhance the system, ensuring that users receive messages when they reconnect. These improvements would position the application for broader usage and align it with modern communication tools.

7.5 Final Thoughts

In conclusion, the LAN-based chat application met its primary objectives by providing a reliable and responsive platform for real-time communication. The use of Java Swing for the interface, TCP sockets for communication, and multi-threading for

performance optimization demonstrated the integration of multiple programming concepts in a cohesive project. Challenges related to concurrency management, disconnections, and input validation were successfully addressed, ensuring the system's stability and usability.

The project also provided valuable insights into network programming and client-server architectures, preparing the foundation for more advanced communication systems. While the application currently meets the requirements for LAN-based usage, future enhancements could extend its functionality and scope, transforming it into a comprehensive messaging platform. This project serves as a solid learning experience in developing distributed systems and highlights the importance of design, error management, and user experience in building practical software solutions.

Bibliography

[1] Ibrahim Muhammed Abba, Umapathy Eaganathan, Norshakirah Ab Aziz, and Janet Gabriel. Lan chat messenger (lcm) using java programming with voip. In *2013 International Conference on Research and Innovation in Information Systems (ICRIIS)*, pages 428–433. IEEE, 2013.

[2] Auth0. Java jwt, 2024.

[3] Gaston C Hillar. *MQTT Essentials-A lightweight IoT protocol*. Packt Publishing Ltd, 2017.

[4] Benjamin Kommey, Elvis Tamakloe, Joseph Boateng, Elijah Ocupualor, John Ayarma, B Kommey, E Tamakloe, J Boateng, E Ocupualor, and J Ayarma. Lan multimedia messaging application. pages 104–119, 12 2022.

[5] OASIS. Mqtt version 5.0, 2019. Accessed: 2024-10-21.

[6] Oracle. Java platform, standard edition (java se). *docs.oracle.com*, 8(1):all, 2014.

[7] Oracle. Package javax.swing. *docs.oracle.com*, page all, 2014.